

All paths are relative to the AzureFable project. Code is most important in sections 3, 5, 6, and 7. Interface screens are best shown in sections 1 and 8.

1. Introduction

- Open:** Run the application, or open Views/MenuView.xaml.
- Show:** Show the game title and the main menu buttons.
- Say:** This is the desktop 2D maze game Azure Fable. It is created in C# and WPF.

2. Goals and Tasks

- Open:** Show Solution Explorer.
- Show:** Show the main folders: Models, Services, Views, ViewModels, Assets.
- Say:** The project is divided into separate parts, so every part has its own responsibility.

3. Overview of Algorithms and Methods

- Open:** Open Models/Maze.cs and Services/GameEngine.cs.
- Show:** Show CanEnter, MoveHero, CheckItemInteraction, and CheckEnemyInteraction.
- Say:** Maze checks if a cell can be entered. GameEngine controls hero movement, item interaction, and enemy collision.

4. Application Structure

- Open:** Open ViewModels/GameViewModel.cs and Views/GameView.xaml.
- Show:** Show that GameViewModel gives data to the screen, and GameView.xaml displays cells, objects, hearts, and key status.
- Say:** This shows the MVVM structure: the View displays the interface, and the ViewModel connects the interface with game logic.

5.1 Inheritance

- Open:** Open Models/GameObject.cs, Unit.cs, Hero.cs, Enemy.cs, and Item.cs.
- Show:** Show the chains GameObject -> Unit -> Hero / Enemy and GameObject -> Item -> Key / Heart / Portal.
- Say:** Inheritance helps avoid code duplication, because common fields and methods are placed in parent classes.

5.2 Polymorphism

- Open:** Open Models/Key.cs, Heart.cs, Portal.cs, Ghost.cs, and StandingEnemy.cs.
- Show:** Show the same methods Interact() and Move(), but with different implementations.
- Say:** The same method is called, but different classes have different behavior.

5.3 Abstraction

- Open:** Open Models/GameObject.cs, Cell.cs, Unit.cs, and Item.cs.
- Show:** Show abstract classes and common properties: X, Y, IsActive, Name, ImagePath.
- Say:** These abstract classes describe common features, not one exact object.

5.4 Encapsulation

- Open:** Open Models/GameObject.cs and Models/Unit.cs.
- Show:** Show private setters for X, Y, and Health, and the methods SetPosition, MoveBy, Heal, and TakeDamage.
- Say:** The object state cannot be changed directly. It changes only through controlled methods.

6. Interfaces

- Open:** Open Models/IInteractable.cs, Models/IMaze.cs, and Services/IGameEngine.cs.
- Show:** Show method signatures, for example Interact, CanEnter, Start, Pause, Resume, and MoveHero.
- Say:** Interfaces describe what a class must do, but not how it does it. This makes the code easier to extend.

7. Architectural Decisions

- Open:** Open ViewModels/ViewModelBase.cs, RelayCommand.cs, Services/GameEngineFactory.cs, and Services/GameEngine.cs.
- Show:** Show INotifyPropertyChanged, ICommand, GameEngineFactory, and DispatcherTimer.
- Say:** These classes support MVVM: ViewModel updates the interface, commands process buttons, and the timer moves enemies.

8. Interface Demonstration

- Open:** Run the application.
- Show:** Show main menu, settings, game screen, help, pause, win screen, or game over screen.
- Say:** Now I will show how these classes work in the actual interface.

9. Conclusions

- Open:** Do not open new code. Keep the game screen or return to GameObject.cs.
- Show:** Briefly remind the committee that the project uses abstraction, encapsulation, inheritance, and polymorphism.
- Say:** The main result is a working 2D maze game where OOP is used as the base of the program structure.

10. Completion

- Open:** Do not open code.
- Show:** Finish calmly.
- Say:** This is the end of my report. Thank you for your attention.